```
/* Question 1:
You are provided with the following dataset containing 15 observations. The dataset
includes a numerical variable score and a grouping variable group (with two groups:
A and B). The data is created using a DATA step in SAS.

a) One-Sample T-Test: Use SAS to conduct a one-sample t-test on the variable score to
test the null hypothesis H0 : μ = 10 against the alternative that the mean is different
from 10.

b) Two-Sample T-Test: Use SAS to perform a two-sample t-test comparing the score values
between Group A and Group B. The null hypothesis for this test is: H0 : μA = μB */

data sample;
input id group $ score;
datalines;
1 A 12
2 A 9
3 A 11
4 B 8
5 A 10
6 B 12
7 B 9
8 A 10
9 B 7
10 B 11
11 A 13
12 B 10
13 A 11
14 B 9
15 A 12
;
run;

/* a) For our one-sample t-test: */

proc ttest h0 = 10 plots(showh0) sides = 2 alpha = 0.1;
   var score;
run;


/* The PROC TTEST statement straightforwardly works through our process of performing a
t-test. The option H0 specifies the null value, while SIDES specifies the number of sides
and direction. The ALPHA option is equivalent to 1 minus the confidence level, so in this
case (since alpha was not specified), I chose a confidence level of 90%. PLOTS is an addi-
tional option that generates various plots and includes the null hypothesis value within
it, adding a visual indicator with which to compare our actual sample mean.

This PROC TTEST procedure automatically generates various statistics to observe. We see
that the number of observations is equal to 15, with a mean of 10.2667 and a standard de-
viation of 1.6676. The standard error is equal to 0.4306, and the data ranges from values
7 to 13. The 90% confidence level regarding the mean spans from 9.5083 to 11.025, while the
90% confidence level regarding the standard deviation spans from 1.2821 to 2.4342.

The most relevant results involve the actual t-value computed, as well as the p-value. The
t-value under this null hypothesis is 0.62. For a 90% confidence interval, the critical
value is around 1.645. Because the absolute value of our t-value is less than the critical,
we fail to reject the null hypothesis; we do not have enough statistically significant evi-
dence to conclude that the null hypothesis is false. The p-value is an additional metric
confirming this. Because it is much larger than any standard significance level (0.05 or 0.1),
we fail to reject the null.


b) For our two-sample t-test: */

proc ttest data = sample plots(showh0) sides = 2 alpha = 0.1;
    class group;
```

```
    var score;
run;


/* Here, we utilize the same statement (PROC TTEST), this time noting the dataset we are
using. The added option CLASS tells SAS we are comparing the two groups, while (again)
VAR score lets SAS know what variable is being tested.

First, we see that group A consists of 8 observations while group B consists of 7. The means
for each group are 11 (A) and 9.4286 (B), while the standard deviations are 1.3093 (A) and
1.7182 (B). The standard errors for groups A and B, respectively, are 0.4629 and 0.6494. Group
A has a range of 9 to 13, while group B has a range of 7 to 12. The 90% confidence level re-
garding the mean for Group A spans from 10.123 to 11.877, and the 90% confidence level regard-
ing the mean for Group B spans from 8.1666 to 10.6905. Additionally, we see that the 90% confi-
dence level on the standard deviation for Group A ranges from 0.9236 to 2.353, while for Group
B it ranges from 1.1861 to 3.2912. We might also note that the difference in the means is 1.5714.

For both the Pooled and Satterthwaite methods, the t-value is larger than the threshold of 1.645
(Pooled t-value = 2.01, Satterthwaite t-value = 1.97). Moreover, the corresponding p-values are
0.0659 and 0.0741, both of which are below 0.1. Therefore, at the alpha = 0.1 level, we would
reject the null hypothesis; there is evidence of a significant difference in the mean scores
between both groups.


QUESTION 2:
Given the following data, run the regression analysis to model Y as a function of the four pre-
dictors. Discuss your results. */

data regression;
input id x1 x2 x3 x4 y;
datalines;
1 1 2.0 10 15 8.5
2 2 2.5 11 14 11.5
3 3 3.0 12 16 16.0
4 4 3.5 9 15 17.5
5 5 4.0 14 12 22.0
6 6 4.5 13 14 25.0
7 7 5.0 10 16 29.5
8 8 5.5 11 15 31.5
9 9 6.0 13 12 37.0
10 10 6.5 12 13 39.5
11 11 7.0 15 15 42.5
12 12 7.5 10 16 47.0
13 13 8.0 11 14 49.0
14 14 8.5 13 12 54.5
15 15 9.0 14 13 57.0
16 16 9.5 9 15 60.0
17 17 10.0 10 14 64.5
18 18 10.5 12 13 66.5
19 19 11.0 11 15 72.0
20 20 11.5 13 12 74.5
;
run;


proc reg data = regression;
    model y = x1 x2 x3 x4;
    title 'Multiple Regression Model: Y vs. X1, X2, X3, and X4';
run;
quit;


/* The PROC REG starts the regression procedure by defining which dataset will be used and
the model we are running. The RUN and QUIT statements execute and exit the procedure.

The first thing to observe that really stands out is that one of our predictor variables, X2,
was dropped from the model because it was a perfect linear combination of the intercept and
```

```
X1. Thus, its coefficient was set to 0, as it did not add any more information to our model.

We see the F-value is extremely large at 4482.74 (with Pr > F less than 0.0001), telling us
that this regression model explains a vast amount of the variation in our dependent variable.
This is also confirmed by the high R-squared (and adjusted R-squared) value of 0.9988 (0.9986),
essentially saying that 99.88% of the variation in Y can be explained by our predictors.

Interestingly, looking at the parameter estimates table, we see the only one with any real sig-
nificance is X1 (coefficient of 3.49763, standard error of 0.03173), with a p-value < 0.0001.
The intercept (4.05424, standard error = 3.50522) had a t-value of 1.16, less than our desired
threshold of 1.645, and a p-value of 0.2644, greater than our desired threshold of 0.1. X3
(coefficient = 0.03384, standard error = 0.12788) and X4 (coefficient = 0.00722, standard error
= 0.16481) were even worse, with t-values of 0.26 and 0.24, respectively, and p-values of 0.7947
and 0.9656, respectively.

We also see that in the DF column that the intercept and X1 are both categorized as biased, prob-
ably because the model had redundancy in X2; Since the coefficients of the intercept and X1 could
no longer be uniquely estimated in the presence of collinearity, they were both flagged. Moreover,
(as stated previously), X2 was eliminated from the model, which meant it had 0 degrees of freedom.


QUESTION 3:
Given the following data, and using PROC SQL in SAS, write a query to:

1. Filter the dataset to include only observations where group equals 'A'.
2. Compute the mean (average) of the variable score for this subset. */
```

```sql
data mydata;
input id group $ score;
datalines;
1 A 10
2 B 12
3 A 15
4 A 14
5 B 13
6 A 9
7 A 11
8 B 12
9 A 10
10 B 16
11 A 13
12 A 12
;
run;

/* For both parts 1 and 2 */
```

```sql
proc sql;
   select mean(score) as average_score
   from mydata
   where group = 'A';
quit;


/* PROC SQL is an alternative to other SAS procedures, allowing one to write standard SQL code
to query/manipulate datasets. In this case, we are simply selecting (returning) the average
score of our dataset for the observations where the group is equal to A. We get a value of 11.75.


QUESTION 4:
Given the following data, and using PROC SQL in SAS, write a query to join the two datasets on
the common variable dept_id and produce an output that includes Employee ID (emp_id), Employee
Name (name), Department Name (dept_name), and Salary (salary). */
```

```sql
data employees;
input emp_id $ name $ dept_id $ salary;
datalines;
```

```
E001 John D01 55000
E002 Mary D02 60000
E003 Bill D01 58000
E004 Susan D03 62000
E005 Tom D02 54000
;
run;
data departments;
input dept_id $ dept_name $;
datalines;
D01 Sales
D02 Marketing
D03 IT
;
run;


proc sql;
    select e.emp_id, e.name, d.dept_name, e.salary
    from employees as e, departments as d
    where e.dept_id = d.dept_id;
quit;


/* Similarly to question 3, we just use SQL code within this PROC SQL statement to return the
specified columns and their values. It is somewhat different than how it would be accomplished
in SQL because in SQLite we would use JOIN to connect the tables; here, we specify that we're
selecting from both tables where the dept_id is the same.

In this case, the three department ids are present for all observations. The table returned to
us shows the combined relevant information.


QUESTION 5:
Use the data from https://www.kaggle.com/datasets/vishesh20/indian-food-data-protein-and-other-macros.
Using either, python/julia/R/SAS:

1. Write a query to select the top 4 rows with the highest protein content.
2. Write a query to group the data by Food Class and calculate the average, maximum, and minimum pro-
tein value for each group.
3. Write a query to count the number of rows where the serving size is greater than 200 and the cost
is greater than 50.


Since I am using the cloud for this assignment, I first had to upload the downloaded dataset to the
cloud. I renamed it to "indian_food". Then using the PROC IMPORT statement below, I was able to im-
port the file into a SAS dataset. */

proc import datafile = '/home/u64207004/indian_food.csv'
    out = indian_food
    dbms = csv
    replace;
    guessingrows = max;
    getnames = yes;
run;

/* For part 1, we will utilize the PROC SQL statement again. Because we are only wanting four rows with
the highest protein content, we must use the OUTOBS option, which limits the number of observations
returned. With this option, we can simply select everything from the csv file, listed in descending
order by the variable "Protein (g)"" (ensuring that the observations with the highest protein end up at
the top, and thus combined with the OUTOBS option, we can obtain only the top four observations). To
descend by the variable, we have to use double quotes and an 'n' suffix. */

proc sql outobs = 4;
   select *
   from indian_food
   order by "Protein (g)"n desc;
```

```
quit;

/* From this, we see that the four food items with the highest protein content were Grilled Chicken with
Salad (25 grams of protein), Grilled Fish with Salad (22 grams of protein), Mutton Curry with Rice (20
grams of protein), and Chicken Curry with Rice (18 grams of protein).


For part 2 */

proc sql;
    select "Food Class"n,
           avg("Protein (g)"n) as avg_protein,
           max("Protein (g)"n) as max_protein,
           min("Protein (g)"n) as min_protein
    from indian_food
    group by "Food Class"n;
quit;

/* Here, we observe that the average protein, maximum protein, and minimum protein listein in a breakfast
recipt are 5.88, 10, and 4, respectively. For dinner, the average, minimum, and maximum were 9.76, 25,
and 4, while for lunch those numbers are 10.28, 20, and 6. Moreover, we see the protein values for the nuts
food class are 2.78, 7. and 0.5, and the protein values for the snacks food class are 3.12, 6, and 1.


For part 3 */

proc sql;
    select count(*) as high_svgsize_cost
    from indian_food
    where "Serving (g)"n > 200 and "Cost (INR)"n > 50;
quit;

/* Thus, we see that the number of rows where the serving size is greater than 200 and the cost is greater
than 60 is fourteen rows.


QUESTION 6:
On the following data, perform a join on these three tables on the common_id column, and report the out-
put. */

data books1;
infile datalines dlm = ' ' dsd;
input book_id $ title :$50. author :$50.;
datalines;
B001 "The Dawn" "Alice Smith"
B002 "Mystic River" "Bob Jones"
B003 "Hidden Secrets" "Cara Lee"
B004 "The Journey" "David Brown"
B005 "Whispers in the Wind" "Eva Green"
;
run;

data books2;
infile datalines dlm = ' ' dsd;
input book_id $ copies_sold region :$50.;
datalines;
B003 5000 "North America"
B001 12000 "Europe"
B006 3000 "Asia"
B007 1500 "South America"
B008 10000 "Africa"
;
run;

data books3;
input book_id $ rating review_count;
datalines;
```

```
B001 4.5 350
B009 3.8 180
B010 4.0 220
B011 5.0 150
B012 4.2 300
;
run;

/* For the first two datasets we create with the DATA statement, we need to specify the delimiter,
telling SAS to handle the data as if it's delimited by spaces but also to treat quoted strings using dsd
(which handles delimiter-separated data with quoted strings). The :$50. modifier ensures that the whole
string inside the quotes is read correctly.

To perform the join: */

proc sql;
    select books1.book_id, title, author, copies_sold, region, rating, review_count
    from books1, books2, books3
    where books1.book_id = books2.book_id
    and books2.book_id = books3.book_id;
quit;

/* Here, because we are trying to return observations with a common id within each table. Using PROC
SQL, I specified that every column should be returned. Since the book id is common across all tables,
I also only included the 'book_id' column from the first table so it wouldn't be repeated three times.
The WHERE statement tells SAS that all the book ids need to be identical in order to return information.
We end up with one result, where the book id is equal to B001, as this is the only common id within
all three tables. This is the book "The Dawn" by Alice Smith, having sold 12000 copies in Europe, where
the book attained a rating of 4.5 out of 350 reviews.


QUESTION 7:
Using the data from https://www.kaggle.com/datasets/emmanueltugbeh/northwind-orders-and-order-details?
select=northwind_orders.csv

The Northwind Orders and Order Details dataset contains two related CSV files:
• northwind_orders: This table contains general order information including OrderID, OrderDate,
ShipCountry, etc.
• northwind_order_details: This table includes detailed information about each order such as ProductID,
UnitPrice, Quantity, and Discount.

1. Perform an inner join between the orders and order details tables. Report on the results.
2. Produce a query that first aggregates the order details to calculate the total amount for each order. */


proc import datafile = '/home/u64207004/northwind_orders.csv'
    out = northwind_orders
    dbms = csv
    replace;
    guessingrows = max;
    getnames = yes;
run;

proc import datafile = '/home/u64207004/northwind_order_details.csv'
    out = northwind_order_details
    dbms = csv
    replace;
    guessingrows = max;
    getnames = yes;
run;


/* We can accomplish this first task with the following query. We simply select everything from both
tables where there is a matching order_id in both tables.  */

proc sql;
  select *
```

```
  from northwind_orders no
  inner join northwind_order_details nod on no.order_id = nod.order_id;
quit;

/* Looking at the results, we see there are quite a bit of observations. Some rows, despite having the
matching id specified in both tables, are missing some values, like in the 'ship_region' column.

To accomplish the second task: */

proc sql;
  select order_id, sum(quantity * unit_price * (1 - discount)) as total_order_amount
  from northwind_order_details
  group by order_id;
quit;


/* With this, we select both the order ID and calculate the total amount spent for each order ID. The
total amount was found by multiplying the number of units purchased by the price of each unit, and then
multiplying that value by 1 minus the discount rate (if there was no discount rate to be applied, then
it would simply be the quantity of units multiplied by the unit price). Note that no join function need
be used for these two tables because all the relevant variables were found in the northwind_order_details
dataset.
```